

Step 1: Open Ubuntu

Paste:

```
cd ~/photonic-device-meep-simulation
```

```
cat > waveguide.py <<'PY'
import meep as mp
import numpy as np
import matplotlib.pyplot as plt
```

```
cell_x = 16
cell_y = 8
resolution = 20
```

```
waveguide_width = 1
waveguide_epsilon = 11.56
```

```
cavity_x = 1.6
cavity_y = 1.5
```

```
outer_radius = 1.0
inner_radius = 0.65
```

```
outer_epsilon = 14.44
inner_epsilon = 1.0
```

```
source_frequency = 0.15
source_width = 0.10
```

```
frequencies = np.linspace(0.10, 0.20, 100)
```

```
def make_geometry():
```

```
    return [
```

```
        mp.Block(
            center=mp.Vector3(0, 0, 0),
            size=mp.Vector3(
                mp.inf,
                waveguide_width,
                mp.inf
            ),
```

```

        material=mp.Medium(
            epsilon=waveguide_epsilon
        )
    ),

    mp.Cylinder(
        radius=outer_radius,
        center=mp.Vector3(
            cavity_x,
            cavity_y,
            0
        ),
        material=mp.Medium(
            epsilon=outer_epsilon
        )
    ),

    mp.Cylinder(
        radius=inner_radius,
        center=mp.Vector3(
            cavity_x,
            cavity_y,
            0
        ),
        material=mp.Medium(
            epsilon=inner_epsilon
        )
    )
]

```

```
def run_simulation(geometry):
```

```

    sources = [

        mp.Source(
            mp.GaussianSource(
                source_frequency,
                fwidth=source_width
            ),
            component=mp.Ez,
            center=mp.Vector3(-6, 0, 0),
            size=mp.Vector3(
                0,

```

```

        waveguide_width,
        0
    )
)
]

sim = mp.Simulation(
    cell_size=mp.Vector3(
        cell_x,
        cell_y,
        0
    ),
    geometry=geometry,
    sources=sources,
    resolution=resolution,
    boundary_layers=[
        mp.PML(1.0)
    ]
)

flux_region = mp.FluxRegion(
    center=mp.Vector3(6, 0, 0),
    size=mp.Vector3(
        0,
        waveguide_width,
        0
    )
)

flux = sim.add_flux(
    source_frequency,
    source_width,
    100,
    flux_region
)

sim.run(
    until=200
)

return np.array(
    mp.get_fluxes(flux)
)

```

```
print("Running reference simulation...")
```

```
reference_geometry = [
```

```
    mp.Block(  
        center=mp.Vector3(0, 0, 0),  
        size=mp.Vector3(  
            mp.inf,  
            waveguide_width,  
            mp.inf  
        ),  
        material=mp.Medium(  
            epsilon=waveguide_epsilon  
        )  
    )  
]
```

```
reference_flux = run_simulation(  
    reference_geometry  
)
```

```
print("Reference simulation complete.")
```

```
print("Running cavity simulation...")
```

```
cavity_flux = run_simulation(  
    make_geometry()  
)
```

```
print("Cavity simulation complete.")
```

```
transmission = (  
    cavity_flux /  
    reference_flux  
)
```

```
minimum_index = np.argmin(  
    transmission  
)
```

```
minimum_transmission = (  
    transmission[minimum_index]  
)
```

```

resonance_frequency = (
    frequencies[minimum_index]
)

print()
print("=====")
print("TRANSMISSION RESULTS")
print("=====")
print(
    f"Minimum transmission: "
    f"{minimum_transmission:.4f}"
)
print(
    f"Frequency: "
    f"{resonance_frequency:.5f}"
)

np.savetxt(
    "transmission_results.csv",
    np.column_stack(
        (
            frequencies,
            transmission
        )
    ),
    delimiter=",",
    header="frequency,transmission",
    comments=""
)

plt.figure(
    figsize=(8, 5)
)

plt.plot(
    frequencies,
    transmission
)

plt.xlabel("Frequency")
plt.ylabel("Transmission")

plt.title(

```

```
    "Photonic Cavity Transmission Spectrum"  
)  
  
plt.grid(True)  
  
plt.tight_layout()  
  
plt.savefig(  
    "transmission_spectrum.png",  
    dpi=200  
)  
  
plt.close()  
  
print("Saved transmission_spectrum.png")  
PY
```

```
cat > position_sweep.py <<'PY'
import meep as mp
import numpy as np
import matplotlib.pyplot as plt
```

```
cell_x = 16
cell_y = 8
resolution = 20
```

```
waveguide_width = 1
waveguide_epsilon = 11.56
```

```
cavity_x = 1.6
```

```
outer_radius = 1.0
inner_radius = 0.65
```

```
outer_epsilon = 14.44
inner_epsilon = 1.0
```

```
positions = [
    1.1,
    1.3,
    1.5,
    1.7,
    1.9
]
```

```
frequencies = np.linspace(
    0.10,
    0.20,
    100
)
```

```
def run_simulation(geometry):
```

```
    sources = [
```

```
        mp.Source(
            mp.GaussianSource(
                0.15,
                fwidth=0.10
            ),
            component=mp.Ez,
            center=mp.Vector3(-6, 0, 0),
            size=mp.Vector3(
                0,
                waveguide_width,
                0
            )
        )
    ]
```

```
    sim = mp.Simulation(
        cell_size=mp.Vector3(
            cell_x,
            cell_y,
            0
        ),
        geometry=geometry,
        sources=sources,
        resolution=resolution,
        boundary_layers=[
            mp.PML(1.0)
        ]
    )
```

```
    flux_region = mp.FluxRegion(
        center=mp.Vector3(6, 0, 0),
        size=mp.Vector3(
            0,
            waveguide_width,
            0
        )
    )
```

```
    flux = sim.add_flux(
```

```

        0.15,
        0.10,
        100,
        flux_region
    )

    sim.run(
        until=200
    )

    return np.array(
        mp.get_fluxes(flux)
    )

reference_geometry = [

    mp.Block(
        center=mp.Vector3(0, 0, 0),
        size=mp.Vector3(
            mp.inf,
            waveguide_width,
            mp.inf
        ),
        material=mp.Medium(
            epsilon=waveguide_epsilon
        )
    )
]

print("Running reference simulation...")

reference_flux = run_simulation(
    reference_geometry
)

print("Reference complete.")

results = []
spectra = []

for y in positions:

```

```
print(  
    f"Running cavity position y = {y}"  
)
```

```
geometry = [
```

```
    mp.Block(  
        center=mp.Vector3(0, 0, 0),  
        size=mp.Vector3(  
            mp.inf,  
            waveguide_width,  
            mp.inf  
        ),  
        material=mp.Medium(  
            epsilon=waveguide_epsilon  
        )  
    ),
```

```
    mp.Cylinder(  
        radius=outer_radius,  
        center=mp.Vector3(  
            cavity_x,  
            y,  
            0  
        ),  
        material=mp.Medium(  
            epsilon=outer_epsilon  
        )  
    ),
```

```
    mp.Cylinder(  
        radius=inner_radius,  
        center=mp.Vector3(  
            cavity_x,  
            y,  
            0  
        ),  
        material=mp.Medium(  
            epsilon=inner_epsilon  
        )  
    )  
)
```

```
]
```

```

cavity_flux = run_simulation(
    geometry
)

transmission = (
    cavity_flux /
    reference_flux
)

spectra.append(
    transmission
)

index = np.argmin(
    transmission
)

f_res = frequencies[index]
t_min = transmission[index]

print(
    f"Minimum T = {t_min:.4f}"
)

print(
    f"Resonance frequency = {f_res:.4f}"
)

results.append(
    [
        y,
        f_res,
        t_min
    ]
)

results = np.array(
    results
)

np.savetxt(
    "position_sweep.csv",

```

```

results,
delimiter=",",
header=(
    "y_position,"
    "resonance_frequency,"
    "minimum_transmission"
),
comments=""
)

# Position vs minimum transmission

plt.figure(
    figsize=(8, 5)
)

plt.plot(
    results[:, 0],
    results[:, 2],
    marker="o"
)

plt.xlabel("Cavity y-position")
plt.ylabel("Minimum Transmission")

plt.title(
    "Cavity Position vs Minimum Transmission"
)

plt.grid(True)

plt.tight_layout()

plt.savefig(
    "coupling_sensitivity.png",
    dpi=200
)

plt.close()

# Position vs resonance frequency

```

```

plt.figure(
    figsize=(8, 5)
)

plt.plot(
    results[:, 0],
    results[:, 1],
    marker="o"
)

plt.xlabel("Cavity y-position")
plt.ylabel("Resonance Frequency")

plt.title(
    "Cavity Position vs Resonance Frequency"
)

plt.grid(True)

plt.tight_layout()

plt.savefig(
    "position_resonance.png",
    dpi=200
)

plt.close()

```

Full transmission spectra

```

plt.figure(
    figsize=(8, 5)
)

for i, y in enumerate(positions):

    plt.plot(
        frequencies,
        spectra[i],
        label=f"y = {y:.1f}"
    )

plt.xlabel("Frequency")

```

```
plt.ylabel("Transmission")

plt.title(
    "Transmission Spectrum vs Cavity Position"
)

plt.grid(True)

plt.legend()

plt.tight_layout()

plt.savefig(
    "position_transmission_sweep.png",
    dpi=200
)

plt.close()


print()
print("POSITION SWEEP COMPLETE")
print()
print("Saved:")
print("coupling_sensitivity.png")
print("position_resonance.png")
print("position_transmission_sweep.png")
PY
```

```
cat > radius_sweep.py <<'PY'
import meep as mp
import numpy as np
import matplotlib.pyplot as plt
```

```
cell_x = 16
cell_y = 8
resolution = 20
```

```
waveguide_width = 1
waveguide_epsilon = 11.56
```

```
cavity_x = 1.6
cavity_y = 1.5
```

```
outer_epsilon = 14.44
inner_epsilon = 1.0
```

```
radii = [
    0.6,
    0.8,
    1.0,
    1.2,
    1.4
]
```

```
frequencies = np.linspace(
    0.10,
    0.20,
    100
)
```

```
def run_simulation(geometry):
```

```

sources = [

    mp.Source(
        mp.GaussianSource(
            0.15,
            fwidth=0.10
        ),
        component=mp.Ez,
        center=mp.Vector3(-6, 0, 0),
        size=mp.Vector3(
            0,
            waveguide_width,
            0
        )
    )
]

```

```

sim = mp.Simulation(
    cell_size=mp.Vector3(
        cell_x,
        cell_y,
        0
    ),
    geometry=geometry,
    sources=sources,
    resolution=resolution,
    boundary_layers=[
        mp.PML(1.0)
    ]
)

```

```

flux_region = mp.FluxRegion(
    center=mp.Vector3(6, 0, 0),
    size=mp.Vector3(
        0,
        waveguide_width,
        0
    )
)

```

```

flux = sim.add_flux(
    0.15,
    0.10,
    100,

```

```

        flux_region
    )

    sim.run(
        until=200
    )

    return np.array(
        mp.get_fluxes(flux)
    )

reference_geometry = [

    mp.Block(
        center=mp.Vector3(0, 0, 0),
        size=mp.Vector3(
            mp.inf,
            waveguide_width,
            mp.inf
        ),
        material=mp.Medium(
            epsilon=waveguide_epsilon
        )
    )
]

print("Running reference simulation...")

reference_flux = run_simulation(
    reference_geometry
)

print("Reference complete.")

results = []
spectra = []

for radius in radii:

    inner_radius = 0.65 * radius

```

```

print(
    f"Running radius = {radius}"
)

geometry = [

    mp.Block(
        center=mp.Vector3(0, 0, 0),
        size=mp.Vector3(
            mp.inf,
            waveguide_width,
            mp.inf
        ),
        material=mp.Medium(
            epsilon=waveguide_epsilon
        )
    ),

    mp.Cylinder(
        radius=radius,
        center=mp.Vector3(
            cavity_x,
            cavity_y,
            0
        ),
        material=mp.Medium(
            epsilon=outer_epsilon
        )
    ),

    mp.Cylinder(
        radius=inner_radius,
        center=mp.Vector3(
            cavity_x,
            cavity_y,
            0
        ),
        material=mp.Medium(
            epsilon=inner_epsilon
        )
    )
]

cavity_flux = run_simulation(

```

```

        geometry
    )

    transmission = (
        cavity_flux /
        reference_flux
    )

    spectra.append(
        transmission
    )

    index = np.argmin(
        transmission
    )

    f_res = frequencies[index]
    t_min = transmission[index]

    print(
        f"Minimum T = {t_min:.4f}"
    )

    print(
        f"Frequency = {f_res:.4f}"
    )

    results.append(
        [
            radius,
            f_res,
            t_min
        ]
    )

results = np.array(
    results
)

np.savetxt(
    "radius_sweep.csv",
    results,

```

```
    delimiter=","  
    header=(  
        "radius,"  
        "resonance_frequency,"  
        "minimum_transmission"  
    ),  
    comments=""  
)
```

```
# Radius vs minimum transmission
```

```
plt.figure(  
    figsize=(8, 5)  
)
```

```
plt.plot(  
    results[:, 0],  
    results[:, 2],  
    marker="o"  
)
```

```
plt.xlabel("Outer Cavity Radius")  
plt.ylabel("Minimum Transmission")
```

```
plt.title(  
    "Radius Sweep: Minimum Transmission"  
)
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.savefig(  
    "radius_sweep.png",  
    dpi=200  
)
```

```
plt.close()
```

```
# Radius vs resonance frequency
```

```
plt.figure(  

```

```

    figsize=(8, 5)
)

plt.plot(
    results[:, 0],
    results[:, 1],
    marker="o"
)

plt.xlabel("Outer Cavity Radius")
plt.ylabel("Resonance Frequency")

plt.title(
    "Radius Sweep: Resonance Frequency"
)

plt.grid(True)

plt.tight_layout()

plt.savefig(
    "radius_resonance.png",
    dpi=200
)

plt.close()

```

Full transmission spectra

```

plt.figure(
    figsize=(8, 5)
)

for i, radius in enumerate(radII):

    plt.plot(
        frequencies,
        spectra[i],
        label=f"R = {radius:.1f}"
    )

plt.xlabel("Frequency")
plt.ylabel("Transmission")

```

```
plt.title(  
    "Transmission Spectrum vs Cavity Radius"  
)
```

```
plt.grid(True)
```

```
plt.legend()
```

```
plt.tight_layout()
```

```
plt.savefig(  
    "radius_transmission_sweep.png",  
    dpi=200  
)
```

```
plt.close()
```

```
print()  
print("RADIUS SWEEP COMPLETE")  
print()  
print("Saved:")  
print("radius_sweep.png")  
print("radius_resonance.png")  
print("radius_transmission_sweep.png")  
PY
```

```
cat > fine_radius_sweep.py <<'PY'  
import meep as mp  
import numpy as np  
import matplotlib.pyplot as plt
```

```
cell_x = 16  
cell_y = 8  
resolution = 20
```

```
waveguide_width = 1  
waveguide_epsilon = 11.56
```

```
cavity_x = 1.6  
cavity_y = 1.5
```

```
outer_epsilon = 14.44  
inner_epsilon = 1.0
```

```
radii = [  
    1.10,  
    1.15,  
    1.20,  
    1.25,  
    1.30  
]
```

```
frequencies = np.linspace(  
    0.10,  
    0.20,  
    100  
)
```

```
def run_simulation(geometry):
```

```
    sources = [
```

```
        mp.Source(
            mp.GaussianSource(
                0.15,
                fwidth=0.10
            ),
            component=mp.Ez,
            center=mp.Vector3(-6, 0, 0),
            size=mp.Vector3(
                0,
                waveguide_width,
                0
            )
        )
    ]
```

```
    sim = mp.Simulation(
        cell_size=mp.Vector3(
            cell_x,
            cell_y,
            0
        ),
        geometry=geometry,
        sources=sources,
        resolution=resolution,
        boundary_layers=[
            mp.PML(1.0)
        ]
    )
```

```
    flux_region = mp.FluxRegion(
        center=mp.Vector3(6, 0, 0),
        size=mp.Vector3(
            0,
            waveguide_width,
            0
        )
    )
```

```
    flux = sim.add_flux(
        0.15,
```

```

        0.10,
        100,
        flux_region
    )

    sim.run(
        until=266.675
    )

    return np.array(
        mp.get_fluxes(flux)
    )

reference_geometry = [

    mp.Block(
        center=mp.Vector3(0, 0, 0),
        size=mp.Vector3(
            mp.inf,
            waveguide_width,
            mp.inf
        ),
        material=mp.Medium(
            epsilon=waveguide_epsilon
        )
    )
]

print("Running reference simulation...")

reference_flux = run_simulation(
    reference_geometry
)

print("Reference complete.")

results = []
spectra = []

for radius in radii:

```

```

inner_radius = 0.65 * radius

print(
    f"Running radius = {radius}"
)

geometry = [

    mp.Block(
        center=mp.Vector3(0, 0, 0),
        size=mp.Vector3(
            mp.inf,
            waveguide_width,
            mp.inf
        ),
        material=mp.Medium(
            epsilon=waveguide_epsilon
        )
    ),

    mp.Cylinder(
        radius=radius,
        center=mp.Vector3(
            cavity_x,
            cavity_y,
            0
        ),
        material=mp.Medium(
            epsilon=outer_epsilon
        )
    ),

    mp.Cylinder(
        radius=inner_radius,
        center=mp.Vector3(
            cavity_x,
            cavity_y,
            0
        ),
        material=mp.Medium(
            epsilon=inner_epsilon
        )
    )
]

```

```

cavity_flux = run_simulation(
    geometry
)

transmission = (
    cavity_flux /
    reference_flux
)

spectra.append(
    transmission
)

index = np.argmin(
    transmission
)

f_res = frequencies[index]
t_min = transmission[index]

print(
    f"Minimum T = {t_min:.4f}"
)

print(
    f"Resonance frequency = {f_res:.5f}"
)

results.append(
    [
        radius,
        f_res,
        t_min
    ]
)

results = np.array(
    results
)

np.savetxt(

```

```

    "fine_radius_sweep.csv",
    results,
    delimiter=",",
    header=(
        "radius,"
        "resonance_frequency,"
        "minimum_transmission"
    ),
    comments=""
)

```

```

best_index = np.argmin(
    results[:, 2]
)

```

```

best_radius = (
    results[best_index, 0]
)

```

```

best_frequency = (
    results[best_index, 1]
)

```

```

best_transmission = (
    results[best_index, 2]
)

```

```

print()
print("=====")
print("FINE SWEEP RESULT")
print("=====")
print(
    f"Best radius = "
    f"{best_radius:.2f}"
)
print(
    f"Minimum transmission = "
    f"{best_transmission:.4f}"
)
print(
    f"Resonance frequency = "
    f"{best_frequency:.5f}"
)

```

```
)
```

```
# Fine radius vs minimum transmission
```

```
plt.figure(  
    figsize=(8, 5)  
)
```

```
plt.plot(  
    results[:, 0],  
    results[:, 2],  
    marker="o"  
)
```

```
plt.scatter(  
    best_radius,  
    best_transmission,  
    s=80  
)
```

```
plt.annotate(  
    f"Best radius = {best_radius:.2f}\n"  
    f"Minimum T = {best_transmission:.4f}",  
    xy=(  
        best_radius,  
        best_transmission  
    ),  
    xytext=(  
        10,  
        20  
    ),  
    textcoords="offset points"  
)
```

```
plt.xlabel("Outer Cavity Radius")  
plt.ylabel("Minimum Transmission")
```

```
plt.title(  
    "Fine Radius Optimization"  
)
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.savefig(  
    "fine_radius_sweep.png",  
    dpi=200  
)
```

```
plt.close()
```

```
# Fine radius vs resonance frequency
```

```
plt.figure(  
    figsize=(8, 5)  
)
```

```
plt.plot(  
    results[:, 0],  
    results[:, 1],  
    marker="o"  
)
```

```
plt.xlabel("Outer Cavity Radius")  
plt.ylabel("Resonance Frequency")
```

```
plt.title(  
    "Fine Radius vs Resonance Frequency"  
)
```

```
plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.savefig(  
    "fine_radius_resonance.png",  
    dpi=200  
)
```

```
plt.close()
```

```
# Full fine-radius transmission spectra
```

```
plt.figure(  

```

```

    figsize=(8, 5)
)

for i, radius in enumerate(radII):

    plt.plot(
        frequencies,
        spectra[i],
        label=f"R = {radius:.2f}"
    )

plt.xlabel("Frequency")
plt.ylabel("Transmission")

plt.title(
    "Fine Radius Transmission Spectra"
)

plt.grid(True)

plt.legend()

plt.tight_layout()

plt.savefig(
    "fine_radius_transmission_sweep.png",
    dpi=200
)

plt.close()

print()
print("FINE RADIUS SWEEP COMPLETE")
print()
print("Saved:")
print("fine_radius_sweep.png")
print("fine_radius_resonance.png")
print("fine_radius_transmission_sweep.png")
PY

```

```
python3 waveguide.py  
python3 position_sweep.py  
python3 radius_sweep.py  
python3 fine_radius_sweep.py  
ls *.png  
explorer.exe ALLURPNGFILES
```

```
# Do one at a time
```